

`codex` CLI를 통해 Phase 3를 구현하는 방법과, 이후 `codex` 없이도 동일한 기능을 수행하는 독립 실행형 에이전트로 전환하는 구체적인 과정을 안내해 드리겠습니다.

🌟 Phase 3 개발 로드맵

Phase 3의 목표는 `codex` 라는 강력한 도구의 도움을 받아 프로토타입을 빠르게 완성하고, 이후 이를 로컬 환경에 최적화된 독립 실행형 에이전트로 발전시키는 것입니다.

📝 1단계: Codex CLI를 활용한 프로토타입 개발

`codex` 는 이미 훌륭한 코딩 에이전트이므로, 이를 활용해 Phase 3의 핵심 로직을 빠르게 구현할 수 있습니다.

- **자동화 스크립트 작성:** `codex` 가 반복적인 개발 작업을 수행하도록 프롬프트를 작성합니다.

```
# 예시: codex에게 독립 실행형 에이전트의 뼈대를 작성하도록 지시
codex -p "Create a standalone Python agent that uses Ollama and Qwen to answer questions abo
```

- **핵심 로직 구현:** `codex` 와 대화하며 다음 요소들을 하나씩 구현합니다.
 - **로컬 LLM 연결:** Ollama 서버와 통신하여 Qwen 모델을 호출하는 모듈 (`ollama_client.py`).
 - **도구(Tool) 정의:** SQLite 데이터베이스에 질의를 실행하는 함수 (`query_database`).
 - **에이전트 루프:** 사용자 질문을 받아 LLM이 도구를 호출하고, 결과를 다시 LLM에 전달하여 최종 답변을 생성하는 ReAct 루프.

🏗️ 2단계: 독립 실행형 에이전트로 전환

`codex` 의 도움을 받아 프로토타입이 완성되면, 이제 그 코드를 기반으로 `codex` 없이도 동작하는 **완전한 독립 실행형 에이전트**를 구축합니다.

1. 아키텍처 설계

기본적인 구조는 다음과 같습니다.

- **CLI 인터페이스:** 사용자로부터 자연어 질문을 입력받습니다.
- **에이전트 코어 (LangChain/LangGraph):** 질문을 분석하고, 도구 사용을 계획하고, 실행 결과를 해석합니다.
- **로컬 LLM (Ollama + Qwen):** 에이전트의 두뇌 역할을 합니다.
- **도구 (Tools):** 에이전트가 사용할 수 있는 함수들입니다 (예: `query_sqlite`).
- **지식 계층 (OKF):** `docs/wiki/` 의 OKF 개념을 읽어와 LLM의 컨텍스트에 제공합니다.

2. 기술 스택 및 구현

- **LLM 백엔드:** Ollama를 사용합니다. Ollama는 OpenAI 호환 API를 제공하므로, 기존 코드를 크게 수정하지 않고도 로컬 모델을 사용할 수 있습니다.

```
# Qwen 모델 다운로드 및 실행
ollama pull qwen2.5:7b
ollama serve
```

- **에이전트 프레임워크:** LangChain과 LangGraph를 사용하는 것이 가장 표준적이고 확장성이 좋은 방법입니다. LangGraph는 복잡한 에이전트 워크플로우를 관리하는 데 특화되어 있습니다.
 - **참고 프로젝트:** 이미 유사한 목적의 오픈소스 프로젝트들이 많이 있습니다. `LangGraph Text-to-SQL Agent` , `Local Reasoning Agent` 등을 참고하면 큰 도움이 됩니다.
- **핵심 구현 예시 (LangChain):**

```
from langchain_community.llms import Ollama
from langchain.agents import create_react_agent, Tool, AgentExecutor
from langchain.tools import tool

# 1. LLM 초기화
llm = Ollama(model="qwen2.5:7b")
```

```
# 2. 도구(Tool) 정의
@tool
def query_sqlite(query: str) -> str:
    """SQLite 데이터베이스에 SQL 쿼리를 실행합니다."""
    # ... (SQLite 연결 및 쿼리 실행 로직) ...
    return str(results)

tools = [query_sqlite]

# 3. 에이전트 생성 및 실행
agent = create_react_agent(llm, tools, prompt)
agent_executor = AgentExecutor(agent=agent, tools=tools, verbose=True)

# 4. 사용자 질문에 대한 응답 생성
response = agent_executor.invoke({"input": "VIP 고객은 몇 명이고, 누구야?"})
print(response['output'])
```

- **OKF 통합:** `docs/wiki/` 디렉토리에서 관련 OKF 개념 파일(예: `vip_customer.md`)을 읽어와 LLM 프롬프트에 포함 시킵니다. 이렇게 하면 "VIP"와 같은 비즈니스 용어를 모델이 정확히 이해할 수 있습니다.

3단계: 성능 비교 및 최적화

독립 실행형 에이전트가 완성되면, `codex` (GPT-4 기반)와의 성능 차이를 비교하고 로컬 환경에 맞게 최적화합니다.

1. 성능 비교 지표

- **정확도 (Accuracy):** 동일한 질문 세트에 대한 SQL 생성 및 응답의 정확도를 비교합니다.
- **응답 속도 (Latency):** 질문 입력부터 응답 출력까지의 시간을 측정합니다.
- **자원 사용량 (Resource Usage):** CPU, 메모리, GPU 사용량을 모니터링합니다.

2. 최적화 전략

로컬 환경의 한계를 극복하고 성능을 끌어올리기 위한 전략들입니다.

- **모델 경량화:** 더 작은 모델을 사용합니다. `qwen2.5:7b` 대신 `qwen2.5:3b` 나 `qwen2.5:1.5b`를 테스트해보는 것이 좋습니다.
- **프롬프트 엔지니어링:** 로컬 모델의 특성에 맞게 프롬프트를 최적화합니다. 역할, 지시사항, 출력 형식을 명확히 지정하면 성능이 크게 향상될 수 있습니다.
- **도구 호출 최적화 (Function Calling):** 가능하다면 모델의 Function Calling 기능을 활용합니다. Qwen2.5는 Tool Calling을 지원하므로, 이를 활용하면 에이전트가 더 안정적으로 도구를 선택하고 호출할 수 있습니다.
- **캐싱 (Caching):** 동일한 질문이 반복될 경우를 대비해 **프롬프트 캐싱**이나 **결과 캐싱**을 도입합니다.
- **하드웨어 가속:** 가능하다면 GPU를 활용합니다. Ollama는 CUDA를 지원하므로, NVIDIA GPU에서 훨씬 빠른 추론이 가능합니다.

Phase 3 완료 조건

- ☐ `codex`를 활용한 독립 실행형 에이전트 프로토타입이 `4-KDQE`에서 동작함.
- ☐ 프로토타입이 `codex` 없이 독립적으로 실행됨.
- ☐ "VIP 고객은 몇 명이고, 누구야?" 등 Phase 2의 모든 테스트 질문에 대해 동일한 결과를 출력함.
- ☐ Ollama + Qwen과 `codex` (GPT-4)의 성능 비교 결과가 문서로 정리됨.
- ☐ 로컬 환경에 최적화된 설정(모델, 프롬프트, 캐싱 등)이 적용됨.

이 단계를 따라 진행하시면, `codex`의 강력한 기능을 빌려 KDQE의 핵심을 빠르게 구현하고, 이를 로컬 환경에 최적화된 가벼운 독립 실행형 에이전트로 성공적으로 전환하실 수 있을 것입니다. 각 단계에서 막히는 부분이 있으면 언제든지 질문해주세요!